# Evaluation Methodology

Measure capability, expose failure, and guide improvement

Evaluation turns a promising AI demo into a system that can be responsibly improved and considered for production. A useful evaluation combines objective checks, human judgment, and carefully designed AI-assisted metrics.

## 01 Why Evaluate?

### Quality control for open-ended systems

Foundation models may answer the same prompt in many ways. Quality can depend on correctness, relevance, faithfulness to context, coherence, safety, and format, so a single yes/no label is often insufficient.

### Evaluation has two jobs

- **Mitigate risk:** find where the system is likely to fail before users depend on it.
- **Reveal opportunities:** identify capabilities that can be improved through data, prompts, models, or product changes.

### Evaluation is a system, not a single score

Language-model metrics can inform foundation-model evaluation because foundation models often contain a language-model component. But token-prediction metrics do not measure every user-facing capability; combine them with task-specific tests and product criteria.

## 02 Challenges in Evaluating Foundation Models

### No single ground truth for many tasks

Summaries, explanations, and creative responses can have multiple valid forms. Constructing reliable criteria and reference examples is difficult, especially when model outputs are open-ended.

### Black-box behavior

Hosted models may expose little about their internals, training data, or update history. Evaluation must therefore emphasize observable behavior and repeatable test cases.

### Benchmarks age quickly

As models improve, older benchmarks may stop distinguishing capabilities. The lecture cites GLUE followed by SuperGLUE and evolving MMLU-style benchmarks as examples of the need to refresh evaluation sets.

### "General purpose" is broad

No finite benchmark covers every way people may use a general-purpose model. Choose tests from expected use cases, risks, user populations, and business needs, and update them as those change.

## 03 Language-Model Metrics

### Entropy and cross-entropy

**Entropy** describes uncertainty in a data distribution. **Cross-entropy** measures how much probability a model assigns to the tokens that actually occur; higher cross-entropy means worse next-token prediction on that data.

$$H(P, Q) = H(P) + D_{KL}(P \,||\, Q)$$

$P$ = true data distribution; $Q$ = model distribution

The irreducible uncertainty in the data and the mismatch between model and data distributions both contribute. KL divergence is non-negative.

## Perplexity

Perplexity is an exponential transformation of cross-entropy and is often interpreted as a measure of next-token uncertainty.

> $\text{Perplexity} = 2^{\text{cross-entropy}}$ when measured in bits
> $\text{Perplexity} = e^{\text{cross-entropy}}$ when measured in nats

- Lower perplexity means the model predicts the evaluation text more confidently.
- More structured text or useful context can make prediction easier.
- Perplexity is a proxy for token prediction, not a complete measure of instruction-following or answer quality.

## Bits per character and byte

**BPC** normalizes loss by characters, but character encoding conventions vary. **BPB** normalizes by bytes and can be more comparable across tokenizers when calculated consistently.

These measures describe compression-like predictability of text, not factual correctness by themselves.

## Use perplexity with care

Very low perplexity on a held-out benchmark can be a warning sign of familiarity or possible train-test contamination; it is not proof of leakage. Very high perplexity can flag unusual or out-of-distribution text. Use data audits and other checks to investigate.

Post-training for instruction-following can shift next-token metrics slightly. Judge the intended application behavior, not perplexity alone.

# 04 Exact and Reference-Based Evaluation

## Functional correctness

When outputs can be executed or checked against constraints, use deterministic tests. Code generation is a strong example: run generated code against unit tests and count whether a valid solution is found.

**pass@k:** generate up to k candidates for a problem and measure the fraction of problems for which at least one candidate passes. Larger k can improve the chance of success but increases inference cost and latency.

## Similarity to a reference

Compare a generated answer with one or more reference answers. References can be human-written or model-assisted, but should be reviewed for correctness and coverage.

- **Exact match:** strict and easy to compute; too rigid for many open-ended tasks.
- **Lexical similarity:** edit distance or n-gram overlap measures shared wording.
- **Semantic similarity:** compare embeddings so paraphrases can be close in meaning.

## Reference metrics can miss correct alternatives

A response may be correct yet differ from the reference wording. This is common in translation, summarization, and explanations. Lexical matching can under-score valid answers, and similarity does not guarantee functional correctness. Use multiple references or task-specific checks when possible.

Similarity methods are also useful for retrieval, ranking, clustering, anomaly detection, and deduplication. The metric should match the job being evaluated.

## Multimodal similarity: CLIP-style example

Encode an image and its caption separately, map them into a shared embedding space, then measure similarity. Contrastive training pulls matching image-text pairs together and separates mismatched pairs. This measures alignment in representation space, not every aspect of image understanding or answer quality.

# 05 AI as a Judge

## What it does

An AI judge evaluates generated outputs using a rubric or comparison prompt. It can provide self-critique, compare a candidate to a reference, or choose the better of two candidate responses.

It can be faster and cheaper than expert review at scale and can help produce preference data. However, it is still a model-based, prompt-sensitive measurement.

## Write a judge rubric carefully

1. State the judging task clearly, such as relevance or faithfulness.
2. Define criteria and what counts as good or poor evidence.
3. Specify a consistent output schema and a compact rating scale.
4. Include good and bad examples when they improve calibration.
5. Control sampling settings and periodically compare judgments with humans.

Pairwise preference is often easier for a judge than assigning a finely calibrated absolute score.

## Common limitations

- **Inconsistency:** the same case can receive different scores.
- **Criteria ambiguity:** prompts, models, and scales differ across evaluators.
- **Cost and latency:** judging adds model calls and tokens.
- **Privacy:** sending outputs to an external judge may expose sensitive data.

## Known judge biases

- **Self-preference:** favoring its own responses.
- **Position bias:** favoring the first candidate; swap order and compare.
- **Verbosity bias:** treating longer answers as better, even when they are not.

### Use a layered evaluation strategy

Combine AI judging with exact tests where available and human review for important or ambiguous cases. A small specialized judge may be useful for a narrow criterion. Reward models score prompt-response pairs; reference-based judges compare to examples; preference models choose between candidate responses.

One practical pattern is to use a lower-cost model for broad generation, then judge a sampled subset and audit results with humans. A judge is not an unquestionable ground truth, especially when it is itself the strongest model in the system.

---

## 06 Comparative Evaluation and Model Ranking

### Head-to-head comparison

Show two candidate outputs for the same task and ask a human or AI judge which is better. A **match** is a comparison; a model's **win rate** is the proportion of its matches it wins.

> Win rate = wins / completed matches

Comparisons capture relative preference, but not whether either response meets an absolute quality threshold.

### Comparative evaluation vs. A/B testing

In side-by-side comparison, the evaluator sees both outputs for one task at the same time. In A/B testing, users are assigned to different variants and typically see one version in a given interaction; outcomes are compared across groups or time.

## Challenges at scale

- **Quadratic comparisons:** all pairs among n models require n(n-1)/2 comparisons.
- **Non-transitive preferences:** A beating B and B beating C does not guarantee A beats C, especially when prompts, judges, and sampling differ.
- **Quality control:** public votes can be noisy, uninformed, or manipulated.
- **Efficient matching:** choose comparisons that reduce uncertainty rather than testing every pair equally.

## Pair relative and absolute measures

"B is better than A" does not mean B is good enough. Pair win rates with task-level thresholds, functional tests, safety checks, and cost-benefit analysis before switching models.

Comparisons are valuable because people can often choose between two outputs more consistently than they can assign an objective score to one output.

## Evaluation pipeline takeaway

Build a repeatable pipeline that reflects intended users and failure modes. Keep held-out data protected, refresh benchmarks, measure both quality and cost, investigate suspicious results, and connect offline evaluation to production monitoring.

---

*Prepared from the supplied Chapter 3 transcript. Metrics are summarized with standard interpretations; benchmark-specific claims remain experimental.*